

STREAMTRACK: A Streams-and-Tables Architecture for Scalable and Reconfigurable Multi-Camera Object Tracking

Mahmoud Tayee^a, Mohamed ElHelw^a, Ahmed Awad^{b,c,*}

^aNile University, Giza, Egypt

^bFaculty of Computers and Artificial Intelligence, Cairo University, Egypt

^cThe British University in Dubai, Dubai, The UAE

Abstract

Multi-camera object tracking (MCOT) is typically built as a tightly coupled, monolithic computer-vision pipeline, which makes cross-camera association hard to scale, identity decisions hard to inspect, and the camera network hard to reconfigure. We argue these are problems of *architecture*, not of the vision task, and re-architect MCOT as a *streams-and-tables* system: detections flow as streams, global identities and camera topology are queryable relational tables, and cross-camera association is a continuous, topology-aware join over them. This turns each difficulty into a property of the design: topology-pruned association (scalability), an inspectable identity view (observability), and topology-as-data (zero-downtime reconfigurability). Realised on an embeddable complex-event-processing engine, the architecture reproduces the global-identity assignments of a state-of-the-art batch baseline while delivering a **156×** speedup in the core similarity stage and up to **34×** end-to-end, with a further **4.16×** latency reduction from topology-aware partitioning at nine cameras.

Keywords: Information System Architecture, Data Streams, Multi-camera Tracking, Streams and Tables

1. Introduction

Multi-camera object tracking (MCOT) underpins a growing class of applications, such as urban traffic management [1] and public-space surveillance [2, 3]. The task is to assign a single, consistent global identity to each physical object as it moves across a network of cameras. Such systems must meet not only computer-vision accuracy targets but also operational demands—scaling to more cameras, exposing intermediate decisions for debugging, and adapting to changes in the camera network.

These operational demands are where conventional MCOT systems struggle, and we argue the cause is *architectural* rather than algorithmic. Such systems are typically built as tightly coupled, monolithic computer-vision pipelines in which cross-camera association logic, identity state, and network topology are entangled in bespoke imperative code [4, 5, 6]. This coupling is precisely what makes association hard to scale (matching defaults to all camera pairs), state hard to inspect (identity decisions live in in-memory program variables), and topology hard to change (re-linking cameras means editing code and restarting the pipeline, losing accumulated state). Treating these as vision problems misdiagnoses them;

*Corresponding author

Email addresses: m.mostafa2118@nu.edu.eg (Mahmoud Tayee), melhelw@nu.edu.eg (Mohamed ElHelw), a.gaafar@fci-cu.edu.eg (Ahmed Awad)

student[Done]
cite 2–3

representa-
tive MCOT
application
papers

student[Done]
cite represen-
tative mono-
lithic/MTMC
pipelines,
e.g. AI City
Challenge
entries

they are consequences of how the system is structured.

We address them by re-architecting MCOT as a *streams-and-tables* system. Per-camera detections and tracklet updates flow as append-only *streams*; global identities and the camera-network topology are maintained as mutable, queryable relational *tables*; and cross-camera association is expressed as continuous CQL queries over them, realised as topology-aware stream joins parameterised by a live camera-neighbourhood table. This rests on the stream–table duality from data-stream management [7, 8], and is not specific to vision: we previously showed that the same paradigm transforms a stream engine into a process-orchestration engine [9]. Demonstrating it now on a problem as different as multi-camera perception is evidence that streams-and-tables is a general architectural approach for re-engineering monolithic, stateful pipelines, not a one-off technique.

The reframing is valuable because it turns each operational difficulty into a property obtained by construction. Pruning association to topological neighbourhoods makes it *scalable*; maintaining identity in a relational table makes the system’s state *inspectable*; and representing topology as data makes the network *reconfigurable at runtime*, with no code change or pipeline restart. We realise the architecture on Esper, an embeddable complex-event-processing engine, in a single process—avoiding the serialisation of high-dimensional feature tensors that a distributed dataflow engine would impose.

This paper makes the following contributions:

- **A streams-and-tables re-architecting of MCOT.** We recast a representative monolithic multi-camera tracking pipeline as an operator graph over detection streams and relational state tables (global identities, camera topology), with cross-camera association expressed as continuous queries.
- **Topology-aware association as stream joins.** Cross-camera matching is a parameterised join over a live neighbourhood table, pruning all-pairs comparisons to localised ones and yielding a **4.16×** per-association latency reduction at nine cameras.
- **Topology as reconfigurable data.** Overlapping and transitional camera links are held in a single relational table, enabling zero-downtime reconfiguration of the network without code changes or loss of tracking state.
- **Deterministic, inspectable state management.** Global identity is a continuously materialised, queryable view, with priority-ordered timer queries bounding memory under sustained load.
- **An end-to-end realisation and evaluation on Esper.** On two multi-camera datasets, the architecture reproduces the global-identity assignments of a state-of-the-art batch baseline while delivering a **156×** speedup in the core similarity stage and up to **34×** end-to-end; we decompose these gains across architectural, algorithmic, and runtime factors.

The remainder of the paper is organised as follows. Section 2 defines the problem and the streams-and-tables background. Section 3 presents the architecture and operator model. Sections 4 and 5 detail the single-camera and cross-camera operators. Section ?? describes the SQL/EPL realisation on Esper. Section 6 reports the evaluation. Sections 8 and 9 discuss related work and conclude.

2. Background and Problem Definition

2.1. Multi-Camera Object Tracking (MCOT)

We consider a set of cameras $\mathcal{C} = \{c_1, \dots, c_m\}$. Each camera c produces a stream of detections $\mathcal{D}_c$, where each detection $d \in \mathcal{D}_c$ consists of a bounding box, an appearance embedding vector $f \in \mathbb{R}^d$, and a timestamp t . The goal of MCOT is to assign a *global identity* ID_G to each observation such that all detections of the same physical object across the camera network $\mathcal{C}$ share the same identity over time, producing consistent and continuous identity tracks.

This goal decomposes into two qualitatively different problems. *Single-camera tracking* links detections within one camera’s field of view across consecutive frames—a local, bounded problem solved well by established trackers [10, 11, 12]. The hard part of MCOT is *cross-camera association*: deciding when a tracklet in one camera is the same physical identity as a tracklet in another, despite changes in viewpoint, illumination, scale, and non-overlapping coverage gaps [13, 14]. It is this cross-camera layer—not single-camera tracking—that dominates the computational cost and determines whether the system scales as cameras are added.

Beyond accuracy, deploying MCOT at scale raises three systems-level challenges that academic tracking literature (such as AI City Challenge benchmarks [15]) largely treats as implementation details, while commercial platforms (e.g., NVIDIA Metropolis [16]) provide scalable deployment frameworks but remain closed-source, heavyweight, and static:

- **Scalable association.** Naive cross-camera matching compares tracklets across all camera pairs, so candidate comparisons grow combinatorially with $|\mathcal{C}|$. Real-world camera networks are spatially sparse; most pairs never see the same person in the same time, yet monolithic pipelines do not exploit this structure.
- **Inspectable state.** The association decision depends on a continuously evolving identity state (which tracklet maps to which global ID, last seen where and when). In monolithic implementations, this state is buried in in-memory program variables, making it impossible to query or audit *why* a particular identity assignment was made.
- **Runtime reconfiguration.** Camera networks change, cameras are added, removed, or re-linked as neighbourhoods evolve. When the network topology is *hardcoded* into the association logic, any such change requires editing code and restarting the pipeline, losing accumulated tracking state.

These challenges are not intrinsic to the tracking task; they are consequences of *architecture*. Conventional MCOT systems (such as the challenge baselines [4, 17, 18]) are built as tightly coupled, monolithic computer-vision pipelines in which cross-camera association logic, identity state, and network topology are entangled in bespoke imperative code. This coupling is precisely what makes association hard to scale, state hard to inspect, and topology hard to reconfigure. The remainder of this paper shows that recasting MCOT as a streams-and-tables system (Section 3) dissolves these three challenges by construction: association becomes a topology-parameterised join, identity state becomes a queryable relation, and topology becomes reconfigurable data. Crucially, this architectural shift must preserve the

student[Done,
cite

SORT/DeepS
or the SCT
used in the
baseline

student[Done,
cite MTMC
/ cross-
camera
Re-ID
surveys

student[Done,
confirm
framing
against
AI City
Challenge

/ MTMC
deployment
papers and
cite

student[Done,
cite repre-
sentative
monolithic
MCOT/MTM
pipelines,
e.g. AI City
Challenge
entries, to
anchor the
“monolithic”

functional output of the tracking task: the re-architected system must match the baseline pipeline’s identity assignments exactly, treating accuracy as an invariant rather than a tuning target.

2.2. Streams and Tables

A data stream is a potentially unbounded, append-only sequence of timestamped data items. Such streams are ubiquitous: sensor updates, market ticks, and process execution events are all examples. Two broad classes of systems process them [19]: those performing SQL-like operations (aggregation, filtering, joining) over one or more streams, and those performing complex event recognition [20, 21], which matches patterns across event sequences to derive higher-level events. A unifying conceptual model for both is the Continuous Query Language (CQL) of Arasu et al. [8, 22], which extends relational algebra with a stream type alongside relations (tables). CQL defines four operator families (Figure 1) that move data between the two types: streams to relations (S2R), streams to streams (S2S), relations to relations (R2R), and relations to streams (R2S). The R2R family is ordinary SQL. S2R operators are window operators [23] that group stream elements (typically by time) into a relation; R2S operators emit a relation’s contents back onto a stream. S2S operators are either stateless (e.g., projection) or stateful, the latter including stream-to-stream joins [24] and CER pattern operators [25].

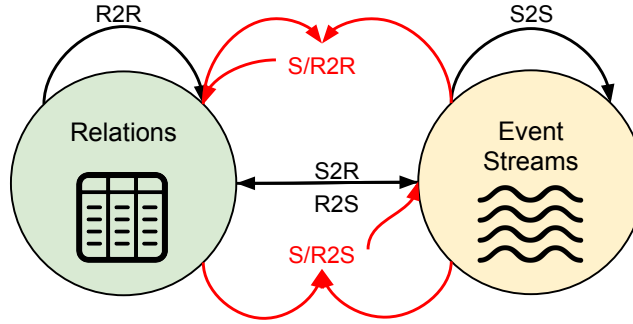


Figure 1: The CQL conceptual model: four operator families transform data between streams and relations [8].

Mixed-input families extend this base. A stream element can be enriched against a relation (a stream–relation join), with the result either updating a relation (S/R2R) or emitted onto a stream (S/R2S). Together, the S2R, R2S, S/R2R, and S/R2S families realise the *stream–table duality* [7]: stream elements upsert into an append-only relation, yielding a queryable snapshot of current state at any instant, while that relation—scanned in arrival order—replays the originating stream. This duality is the conceptual core of our approach; Section 3 instantiates it for our multi-camera tracking.

Table 1 summarises stream-management engines and their support for CQL syntax and operator families. We adopt the operator-family taxonomy from our prior formulation of streams-and-tables for process orchestration [9] and re-target it from process enactment to the perception domain. We select Esper for the reasons established there—full CQL operator support in an embeddable, single-process JVM runtime—which in the MCOT setting additionally avoids serialising high-dimensional feature tensors across distributed workers (Section ??). The generated EPL is otherwise portable to other CQL-compliant engines.

Table 1: CQL-compliant engines: operator-family support analysis.

Engine	CQL Syntax	S2S	S2R	R2R	R2S	S/R2S	S/R2R	Status
<i>True CQL-compliant engines</i>								
Stanford STREAM [26]	Yes	Yes	Yes	Yes	Yes	Partial	Partial	Discontinued
Esper (EPL)	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Active
Oracle Event Processing [27]	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Active
Apache Flink [28]	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Active
Apache Kafka Streams [24]	No	Yes	Yes	Yes	Yes	Yes	Yes	Active
<i>SQL-based streaming (non-CQL compliant)</i>								
Apache Spark Streaming [29]	No	Yes	Yes	Yes	Yes	Yes	Yes	Active
Apache Samza [30]	No	Yes	Yes	Yes	Yes	Yes	Yes	Active
Apache Storm [31]	No	Yes	Yes	Yes	Yes	No	No	Active

2.3. Reference Pipeline

As a concrete, representative instance of the monolithic architecture above, we take a state-of-the-art batch MCOT pipeline (OSC [4]) comprising four stages: per-camera tracklet construction (single-camera tracking), representative feature selection per track, cross-camera re-identification via similarity-based clustering, and a low-confidence detection assignment pass. Our work preserves this functional intent stage-for-stage while re-architecting it as a streams-and-tables operator graph (Section 3), so that tracklet state and identity mappings are maintained in queryable relational tables rather than buried in pipeline code.

3. Architecture

STREAMTRACK instantiates the stream–table duality (Section 2.2) for multi-camera tracking by mapping the three CQL primitives directly onto the tracking problem (Figure 2). *Streams* carry immutable observations—per-camera detections and incremental tracklet updates. *Tables* hold mutable state—the global identity mapping (`GlobalIDTable`) and the camera-network topology (`CameraTopologyTable`). *Continuous queries* bridge them, consuming detection streams, reading and updating these tables, and emitting a derived `GlobalPersonEvent` stream. Casting MCOT in these terms—rather than as a bespoke imperative pipeline—is what makes the system inspectable, reconfigurable, and scalable.

The architecture is organised as a pipeline of continuous operators over detection streams and relational state tables, shown end-to-end in Figure 3. The same operator graph serves a single camera or an arbitrary camera network: per-camera operators (Section 4) produce local tracklet results, and cross-camera operators (Section 5) associate them into global identities. This section specifies *what* each operator is—its inputs, outputs, processing logic, and CQL operator family—while Sections 4 and 5 detail *how* the single- and multi-camera stages are realised.

student[Done]
cite the
exact
baseline
(AI City
Challenge
2024 win-
ning entry /
OSC) here

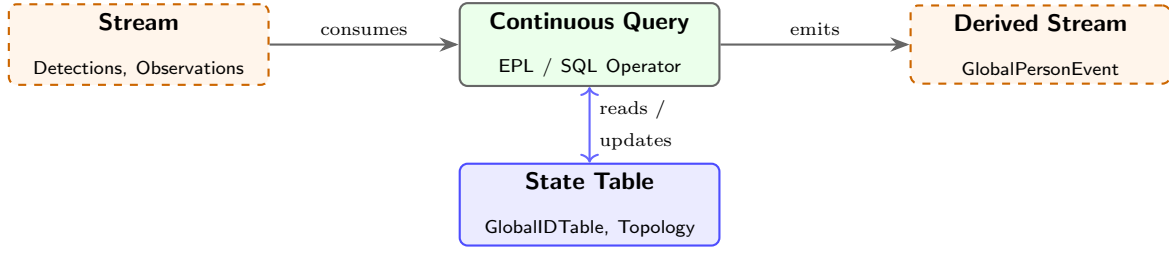


Figure 2: **STREAMTRACK**'s streams-and-tables instantiation of MCOT. Detection streams feed continuous EPL/SQL queries that maintain the `GlobalIDTable` and `CameraTopologyTable` as relational state and emit the derived `GlobalPersonEvent` stream.

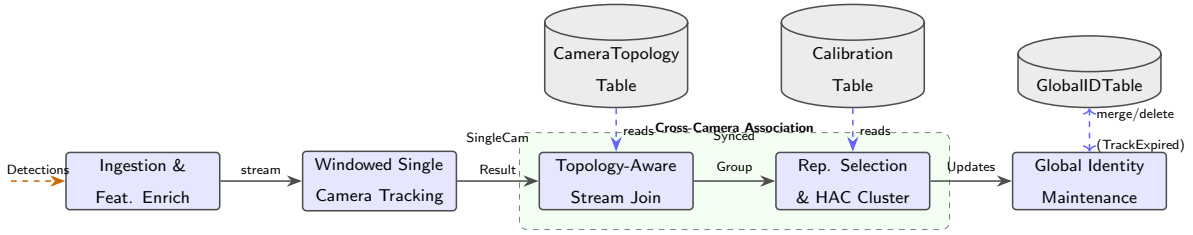


Figure 3: **STREAMTRACK** Operator Graph. Detections arrive as streams from individual cameras. Single-camera tracking is a stateful stream operator. Cross-camera association uses a topology-aware stream join, parameterised by the `CameraTopologyTable`, before performing global appearance clustering. State tables enable memory-safe, time-bounded identity maintenance.

3.1. Operator Model

Every operator in **STREAMTRACK** is specified uniformly by the streams and tables it consumes, the stream or table it produces, its processing logic, and the CQL operator family it belongs to. Table 2 gives this specification for the full pipeline; the streams and state tables it references are defined below.

Streams.

- `embeddingFeature(cameraId, curFrame, timestamp, detectedUsers[])`
- `SingleCameraResult(cameraId, windowIndex, clusterLabels[], featureList[], frameNumbers[], timestamps[], ...)`
- `GlobalPersonEvent(globalId, cameraId, serial, localId, x, y, timestamp, attributes{})`
- `TrackExpiredEvent(globalId)`

State Tables.

- `GlobalIDTable(globalId, lastSeen, attributes{Cameras[], Timestamp, Coordinate})`
- `CalibrationTable(cameraId, homographyMatrix)`
- `CameraTopologyTable(cameraId, neighborId, enabled, overlapping)`

Mapping these operators onto the CQL families makes the duality concrete and shows that each MCOT challenge from Section 2.1 is answered by a specific operator class. Ingestion and feature enrichment is a

student[Done]
Review the
streams
in/out in
Table 2
...)
correct if
necessary

student[Done]
align the
stream
names and
table names
with what in
Table 2

student[Done]
review the
following
part. For
example

Table 2: The **STREAMTRACK** operator model. Each operator is specified by the streams and tables it consumes, what it produces, its processing logic, and the CQL operator family it realises. Vision kernels (Re-ID embedding, homography projection, HAC) are treated as black-box UDFs invoked by the operators; the streams-and-tables layer performs the relational coordination.

Operator	Input (stream / Output table)	Processing logic	CQL
Ingestion & Feature Enrichment	raw detections embeddingFeature (stream)	Project and attach Re-ID embedding	S2S
Single-Camera Tracking	embeddingFeature (stream)	SingleCameraResult (stream) Windowed frame-to-frame association and post-processing (sNMS, warp, short-track exclusion)	S2R → R2S
Topology-Aware Association	SingleCameraResult (stream)	synced cross-camera group (stream) Windowed stream aggregation parameterised by topology	S2R → R2S
Representative Selection	synced group (stream); CalibrationTable	GlobalPersonEvent (stream) Keypoint/centrality selection; homography enrichment and clustering (MCPT)	S/R2S
Global Identity Maintenance	GlobalPersonEvent (stream); GlobalIDTable	updated GlobalIDTable Upsert (MERGE); materialise identity view	S/R2R
Track Expiry & Eviction	GlobalIDTable (timer)	TrackExpiredEvent (stream) Staleness scan; emit and delete stale rows	R2S / R2R

stateless stream-to-stream (S2S) map. Single-camera tracking pairs a stream-to-relation (S2R) window operator with a relation-to-stream (R2S) emission of **SingleCameraResult**. Cross-camera association is realized as a topology-parameterised stream aggregation (S2R → R2S): the **SingleCameraResult** stream is grouped and synchronised across cameras using a windowed time-batch and a dynamic **IN(...)** camera-group filter, with the active query graph automatically redeployed by a reactive topology manager whenever the network topology changes. This directly addresses the *scalable-association* and *reconfiguration* challenges. Global identity maintenance writes each **GlobalPersonEvent** into the **GlobalIDTable** via a stream–relation merge (S/R2R), continuously materialising the identity view—the *inspectable-state* challenge—while a priority-ordered timer query emits **TrackExpiredEvent** (R2S) and evicts stale rows (R2R). The **GlobalIDTable** is thus both a queryable snapshot of identity and a replayable event source: the stream–table duality in operation.

3.2. Execution Semantics

Operators are evaluated continuously as new tuples arrive, i.e., new camera frames. Windows and joins use ingestion timestamps *ts_ingest*. State tables maintain current hypotheses and apply eviction rules based on ingestion-time staleness (e.g., tracklet inactivity).

3.3. Realisation on Esper

The operator model above is engine-agnostic: any CQL-compliant engine (Section 2.2) could host it. We realise it on Esper, whose EPL provides every operator family together with first-class relational tables, and whose embeddable single-process runtime suits the tensor-heavy MCOT setting. Sections 4 and 5 give the per-operator EPL; here we state the engine-level rationale and the conventions shared across all operators.

Why SQL-first on a CEP engine.. Implementing MCOT via SQL/EPL on a complex-event-processing engine such as Esper [32] offers four systems-level advantages over conventional distributed stream processors:

- **Embeddable JVM-local runtime vs. distributed clusters.** Distributed frameworks such as Apache Flink [33] or Spark Streaming run heavy external clusters and incur network serialisation latency when passing high-dimensional visual feature tensors across node boundaries. Esper is a lightweight embeddable library running inside the application process, enabling zero-copy sharing of appearance tensors between tracking operators and relational state in memory.
- **Relational state-table semantics vs. stream-only systems.** Log-processing systems such as Apache Kafka [34] offer only basic state abstractions and lack first-class SQL-like tables with primary keys, indexes, and continuous transactional merges. Esper provides full relational tables, letting us model the active camera neighbourhood and global identity mappings as queryable relations that can be inspected at runtime to audit association decisions.
- **Deterministic memory and tensor management.** Whereas Flink CEP stores state in opaque backends (e.g., RocksDB), Esper allows priority-annotated temporal queries (e.g., `TrackExpiredEvent`) that evict stale `GlobalIDTable` rows and trigger immediate JVM garbage collection of heavy vision tensors after a configurable inactivity window.
- **Zero-downtime topology reconfiguration.** In Flink or Spark, changing the join topology requires stopping the job and redeploying from a savepoint. Esper compiles, deploys, and undeploys EPL statements at runtime, so our reactive loop installs updated topological joins on-the-fly when the `CameraTopologyTable` changes, without interrupting tracking.

Unlike database-oriented video query systems (e.g., MOQL [35], BilVideo [36], SVQL [37]), built for static repositories, and pure pattern-matching CEP systems (e.g., Snoop [38], SASE [39], TESLA [40]), which lack mutable indexed state, Esper uniquely unites stream windowing/joins with queryable relational tables in one engine.

Modular operator design.. The architecture is functionally modular, akin to interchangeable “Lego-like” blocks. By decoupling the temporal stream-routing logic (EPL) from the heavy tracking kernels, the system permits seamless algorithm substitution. Esper handles stream synchronisation, windowing, and topological joining; when a window fills, it triggers a listener that dispatches the collected data to a native tracking operator (Figure 3). A developer can swap the clustering kernel (e.g., replace the default agglomerative clusterer with a density-based or deep classifier) by updating the listener registration at the engine–kernel boundary, evolving the vision logic without redesigning ingestion or synchronisation.

Ordering and jitter under ingestion time.. Ingestion-time semantics avoid the complexity of event-time watermarking. To absorb out-of-order delivery and jitter, the system uses a timeout-bounded aggregation window (e.g., `win:time(...) group by windowIndex having count(*) = numCameras`): the aggregation advances either when all participating cameras have reported for a `windowIndex`, or when the maximum latency expires, guaranteeing a predictable progress rate even if a camera fails to report.

Configuration parameters.. Runtime-tunable parameters are exposed via a `ConfigurationTable`: (i) synchronisation window size (frames), (ii) clustering similarity thresholds, (iii) neighbourhood links in the topology, and (iv) state-eviction TTLs—allowing administrators to tune the accuracy–latency trade-off dynamically.

4. Single-Camera Operators

These operators run independently per camera. They read only per-camera streams, touch none of the cross-camera state tables, and produce the clean `SingleCameraResult` stream that the cross-camera layer (Section 5) consumes. This is the *local* half of the two-problem decomposition introduced in Section 2.1. Each operator below is realised as one or more EPL statements on Esper (Section 3.3).

4.1. Ingestion and Feature Enrichment

This stage ingests per-camera detections and attaches appearance-embedding vectors extracted by a pre-trained Re-ID model, emitting the `embeddingFeature` stream (a stateless S2S map). For cameras where world locations are required, a continuous listener populates the `CalibrationTable` with per-camera homography matrices at ingestion time. The table is subsequently *read* by the native MCPT clustering kernel (Section 5.5), which projects bounding-box foot-points from the 2D image plane to a unified ground-plane coordinate system, enabling spatial gating and distance-based heuristics during cross-camera association. Thus, the `CalibrationTable` is *maintained* at ingestion but *consumed* downstream—consistent with its attribution to the Representative Selection row in Table 2.

Feature extraction happens at the ingestion boundary, before events enter Esper; each enriched detection is then streamed in under a per-camera event type:

```
-- Features are extracted and attached before sending to Esper:
-- e.g., streamEvent(new embeddingFeature(...),
--                               "embeddingFeature_camera_0001");
```

4.2. Single-Camera Tracking as Stateful Stream Processing

Single-camera tracking is a stateful windowed operator ($S2R \rightarrow R2S$). It maintains detection history within a sliding window (e.g., 90–120 frames) to perform frame-to-frame association, aggregating appearance features and keypoint-stability metrics per tracklet, and emits one `SingleCameraResult` event once the tracklet’s window is processed. The window operator is expressed directly in EPL with a length-batch window:

```
-- Stateful windowed single-camera tracking
select detectedUsers, curFrame, timestamp
from embeddingFeature_camera_0001.win:length_batch(90)
```

4.3. Overlap Suppression

Each camera’s detection stream is cleaned by an overlap-suppression step that removes redundant, overlapping bounding boxes so that each physical object yields a single best detection rather than several near-duplicate boxes. This is a standard non-maximum suppression (NMS) operation [41] applied *independently within a single camera*. In our implementation, a sequential NMS variant suppresses temporally and spatially overlapping detections at the ingestion boundary before events enter the tracking window. As a stateless S2S transformation that consumes detections and emits deduplicated ones, it precedes tracking and improves the quality of the tracklets formed downstream. By removing spurious within-camera duplicates, it also indirectly reduces false candidates entering the cross-camera join. Being a detector-side post-processing step, it is applied at the ingestion boundary and has no EPL counterpart of its own. No separate cross-camera duplicate-merge step exists; cross-camera identity resolution is handled entirely by the topology-aware association operators in Section 5.

Disambiguation: overlap suppression vs. field-of-view overlap. Overlap suppression must not be confused with *field-of-view (FoV) overlap*, despite the shared term. Overlap suppression operates on *bounding boxes within a single camera*: it is a detection-level deduplication step. FoV overlap is a *multi-camera deployment property* in which two or more cameras observe the same physical region, possibly from different viewpoints. FoV overlap is a relation *between* cameras, represented explicitly by the `overlapping` attribute of the `CameraTopologyTable` (Section 5.3), where it determines which cameras are grouped into synchronisation joins. The two concepts operate at different levels—pixels within a camera versus visibility across cameras—and are never combined in a single operator.

5. Cross-Camera Operators

These operators reason over two or more cameras. They consume the `SingleCameraResult` stream and read or update the cross-camera relations—`CameraTopologyTable`, `CalibrationTable`, and `GlobalIDTable`—to assign consistent global identities. This is the *global* half of the decomposition, and where the systems-level challenges of Section 2.1 (scalable association, inspectable state, runtime reconfiguration) are answered. As in Section 4, each operator is realised as EPL on Esper (Section 3.3).

student[Done]
cite NMS
/ the
detector’s
box-
suppression
method

5.1. Naive Cross-Camera Matching Cost

Naive cross-camera association compares tracklets across all camera pairs. For m cameras, each with multiple tracklets, exhaustive matching produces a combinatorial explosion of candidate comparisons, which becomes the primary scalability bottleneck as the network grows—the *scalable-association* challenge in concrete form.

5.2. Topology-Aware Candidate Generation

We formulate cross-camera association as a topology-parameterised stream aggregation query synchronised by a `windowIndex` (an $S2R \rightarrow R2S$ operator). The pipeline dynamically builds an `IN(...)` clause corresponding to the explicitly linked cameras of a topological cluster. By restricting comparisons and aggregations to localised camera groups, the candidate count drops sharply—confirmed by our 9-camera benchmark, where topology-aware partitioning yields a $4.16\times$ reduction in per-call latency relative to global matching (Section 6). The localised aggregation is expressed as a time-windowed query whose membership predicate is the topological neighbour set:

```
-- Emits CrossCamCandidates stream (Aggregation Group)
select window( * ) as results
from SingleCameraResult(cameraId in
    ('camera_0001', 'camera_0002')).win:time(2 min)
group by windowIndex
having count(*) = 2;
```

5.3. Camera Topology as a Reconfigurable State Table

A key feature is treating camera neighbourhoods as a relational configuration rather than hardcoded logic. The `CameraTopologyTable` stores the active neighbour graph, where the `overlapping` flag marks pairs of cameras with *shared fields of view* (Section 4.3 contrasts this with within-camera overlap suppression). Administrators update the table at runtime by streaming `CameraTopology` events, and changes take effect for the next window of tracklets. This tables-as-data design supports zero-downtime reconfiguration (adding a camera, disabling a link) without redeploying the association queries—thereby directly answering the *runtime-reconfiguration* challenge.

The topology is declared as a relational table with a composite key, and updated at runtime by an atomic upsert driven by incoming `CameraTopology` events:

```
-- Define relational neighborhood graph
-- (Esper 9 composite primary key syntax)
create table CameraTopologyTable (
    cameraId string, neighborId string, enabled boolean, overlapping boolean,
    primary key(cameraId, neighborId)
);

-- Update topology at runtime via atomic upsert (on-merge)
```

```

on CameraTopology as ct
merge into CameraTopologyTable as target
315 where target.cameraId = ct.cameraId
      and target.neighborId = ct.neighborId
when matched then
      update set target.enabled = ct.enabled, target.overlapping = ct.overlapping
when not matched then
320 insert select ct.cameraId as cameraId,
              ct.neighborId as neighborId,
              ct.enabled as enabled,
              ct.overlapping as overlapping;

```

Critically, the `CameraTopologyTable` is not merely a static configuration store. The system implements
325 a *reactive reconfiguration loop*: each incoming `CameraTopology` event triggers a Connected Components computation over the in-memory neighbourhood graph. When the resulting cluster partition differs from the currently active set of aggregation queries, the system automatically compiles and deploys new EPL statements targeting the updated camera groups, while undeploying the obsolete ones. This enables zero-downtime reconfiguration of the camera network—an operator can add, remove, or reassign camera
330 neighbourhoods at runtime without restarting the pipeline or manually editing query definitions.

5.4. Handling Non-Overlapping Cameras via Transitional Gating

A challenge in real deployments is the existence of isolated (non-overlapping) cameras. If neighbourhoods are modelled purely on overlapping fields of view, a person leaving an isolated camera and entering another is assigned a new global identity, since no active stream join spans both cameras.

335 To resolve this without losing the benefits of topological partitioning, we introduce *transition edges* in the topology graph. Unlike overlapping edges, which group cameras into synchronisation joins, transition edges define directed or bidirectional movement paths between non-overlapping regions. They are stored in the same relational `CameraTopologyTable` alongside overlapping edges, so both overlapping joins and non-overlapping transition boundaries are reconfigurable at runtime by streaming `CameraTopology`
340 events.

For example, a person leaving the isolated view of `camera_12` may transition to `camera_02`, `camera_13`, or `camera_17`. With overlapping groups alone, `camera_12` has no neighbours and any departing track terminates, forcing a new identity on reappearance. Configuring the transitions `12:02,13,17` registers these paths.

345 During global association, before computing appearance similarities between active-window clusters and historical tracks in the `GlobalIDTable`, the system executes a topology-gating check:

$$\text{Re-IDAllowed}(T_H, C_{curr}) = \begin{cases} \text{True} & \text{if LastSeenCamera}(T_H) \in C_{curr} \\ \text{True} & \text{if } \exists c \in C_{curr} \text{ s.t. } (\text{LastSeenCamera}(T_H), c) \in E_{\text{overlap}} \cup E_{\text{transition}} \\ \text{False} & \text{otherwise} \end{cases} \quad (1)$$

where T_H is the historical track, C_{curr} the cameras containing the current cluster, $E_{overlap}$ the overlapping edges, and $E_{transition}$ the transition paths.

When a person leaves **camera_12**, their last-seen camera in the **GlobalIDTable** is set to **camera_12**; when a tracklet appears in **camera_02**, **camera_13**, or **camera_17**, the gate admits the comparison and the global ID is preserved across the non-overlapping gap. All other cameras are bypassed, keeping the search localised and avoiding false positives in distant parts of the network (Figure 4).



Figure 4: Transition topology mapping for isolated cameras. A person leaving the non-overlapping field of view of **Camera 12** can transition to **Camera 02**, **Camera 13**, or **Camera 17**. Transitional gating restricts Re-ID candidate matching to these paths, preventing identity fragmentation across non-overlapping zones.

5.5. Representative Selection

Once cameras are joined and aggregated, the MCPT module selects the top- k representative observations per associated tracklet (an S/R2S enrichment against **CalibrationTable**). Selection uses two keypoint criteria (**keypointTh**, **keypointConditionTh**): (i) presence of high-confidence anatomical landmarks (facial features, limb joints), and (ii) temporal stability of appearance embeddings within the window. This ensures the subsequent Re-ID clustering operates on the most discriminative data available.

5.6. Online Clustering and Global Identity Maintenance

Global identities are maintained in the **GlobalIDTable** (an S/R2R write path). A *person-centric summary* logic reduces event volume: instead of streaming per-frame coordinates, the system aggregates all sightings of a person across the synchronised cluster and emits a single **GlobalPersonEvent** per identity per window, capturing the latest sighting (**Timestamp**, **Coordinate**) and the full camera set (e.g., **Cameras**: [1,2,4]). Clustering itself is performed by a windowed Hierarchical Agglomerative Clustering (HAC) operator that executes at the end of each synchronisation window; because the work is partitioned into topological groups, the clustering load per group stays manageable even under high-throughput streams.

The identity view is maintained by an on-merge upsert keyed by **globalId**:

```
-- Maintains GlobalIDTable with latest state and person-centric summaries
on GlobalPersonEvent as gpe
```

```

merge into GlobalIDTable as gt
where gt.globalId = gpe.globalId
when matched then
    update set gt.lastSeen = gpe.timestamp, gt.attributes = gpe.attributes
375 when not matched then
    insert select gpe.globalId as globalId,
               gpe.timestamp as lastSeen,
               gpe.attributes as attributes;

```

A key operational advantage is deterministic memory management—the *inspectable-state* challenge
380 in practice. Batch trackers often suffer unbounded memory growth from accumulating appearance signatures. Here, when the engine detects a person unseen across all cameras for a configured period (e.g., 2 minutes), a priority-ordered EPL timer emits a `TrackExpiredEvent` (R2S) and then evicts the stale rows (R2R delete), purging the `GlobalIDTable` and garbage-collecting the heavy feature tensors from JVM memory:

```

385 -- Emit TrackExpiredEvent and clean up GlobalIDTable (with Priority handling)
@Priority(10) on pattern [every timer:interval(10000)]
insert into TrackExpiredEvent select globalId from GlobalIDTable
where current_timestamp() - lastSeen > 120000;

390 @Priority(1) on pattern [every timer:interval(10000)]
delete from GlobalIDTable
where current_timestamp() - lastSeen > 120000;

```

5.7. Join Planning and Optimisation

Topology sparsity shrinks the join search space and improves throughput. With m cameras in the
395 global network partitioned into k topology groups of sizes $n_1, \dots, n_k$ ($\sum n_i = m$), the number of pairwise candidate comparisons drops from $\binom{m}{2}$ (all-pairs) to $\sum_{i=1}^k \binom{n_i}{2}$, which is strictly less for $k > 1$. For our 9-camera NVIDIA SmartSpaces deployment partitioned into four groups of sizes $\{2, 2, 2, 3\}$, naive matching produces $\binom{9}{2} = 36$ camera pairs, whereas topology-aware partitioning reduces this to $1 + 1 + 1 + 3 = 6$ pairs—a $6\times$ reduction in cross-camera pair evaluations. Because each pair contributes a quadratic number
400 of tracklet comparisons, the realised speedup at the clustering level is $4.16\times$ (Table 6), with the residual difference attributable to varying per-group tracklet densities. Section 6 details the end-to-end latency measurements.

6. Experimental Evaluation

6.1. Research Questions

- 405 • **RQ1:** Does the streams-and-tables re-architecture preserve tracking quality compared to the baseline pipeline?

- **RQ2:** How much does topology-aware joining reduce candidate comparisons and runtime cost compared to naive all-pairs matching?
- **RQ3:** How robust is the system under runtime topology changes (without restart/reset) in terms of stability and accuracy?
- **RQ4:** How do throughput and latency scale with the number of cameras and detection rates?

We evaluate on both the Woven VisionAI WTS dataset [42] and the NVIDIA SmartSpaces dataset [43], focusing on scenarios with varying camera graph sparsity and topology updates during execution. We focus our quantitative evaluation on systems-level metrics: throughput (windows per second), peak processing latency for the clustering stage, and similarity matrix generation time.

6.2. Baselines

Baselines include: (i) the original batch-oriented baseline pipeline (**Theirs**), (ii) a relational streaming implementation without topology pruning (**Ours-Base**), and (iii) the full optimized pipeline with topology-aware joins (**Ours-Opt**).

6.3. Implementation Parity and Functional Accuracy

Before evaluating performance gains, we first verify that the proposed streaming architecture preserves tracking quality. We evaluate the *Implementation Parity* between **Ours** (StreamTrack) and the reference baseline (**Theirs**) using a serialized cross-implementation mapping method. Functional equivalence is quantified across two pipeline stages using the following metrics:

- **Numerical Parity (Similarity Matrix):** We measure code-level equivalence by calculating the **Mean Absolute Difference (MAD)** between the Re-ID similarity matrices generated by both systems. A near-zero MAD across all windows confirms that our Relational Stream implementation maintains the numerical precision of the batch baseline.
- **Identity Parity (Global ID):** We confirm the final identity assigned to every physical object across the entire scene to verify absolute output consistency.

Table 3 summarizes these results for the Woven WTS Dataset.

Table 3: Implementation Parity: StreamTrack vs. AIC24 Baseline (Woven WTS Dataset).

Pipeline Stage	Metric (Abbreviation)	Parity Result
Re-ID Similarity Matrix	Mean Absolute Difference (MAD)	$< 1 \times 10^{-6}$
Global Tracking	Global ID Assignment Match	100.00%

The metrics characterize different stages of the tracking pipeline: MAD quantifies numerical error (where 0 is perfect) at the Re-ID similarity matrix stage, while the Global ID Assignment Match measures final output consistency across the entire scene. The results confirm that our streaming engine achieves both exact numerical parity ($< 1 \times 10^{-6}$ MAD) and absolute functional parity (100.00% Global ID Match)

with the original batch pipeline. By mapping incoming clusters against a persistent historical consensus of identity state, the stateful `GlobalIDTable` maintains identical track associations. Consequently, all downstream tracking accuracy metrics (IDF1, HOTA) are preserved by construction, as visually affirmed in Figure 5.



Figure 5: Visual comparison of tracking results for NVIDIA SmartSpaces Camera 01. The frame illustrates the parity between the batch baseline (**Theirs**) and our relational streaming implementation (**Ours**), confirming that the performance optimizations do not compromise tracking coverage or identity consistency.

6.4. Comparative Analysis of Clustering Operators

While the streaming database literature often suggests specialized algorithms like CluStream or ClusTree for high-velocity data, our experiments demonstrate that for the specific window sizes encountered in MCOT (50–500 tracklets), Hierarchical Agglomerative Clustering (HAC) is superior in both latency and identity precision. Table 4 summarizes a side-by-side benchmark conducted on the NVIDIA SmartSpaces dataset.

Table 4: Clustering Operator Comparison (NVIDIA SmartSpaces, 50-Window Average)

Algorithm	Avg. Latency (ms)	ID Precision	Overhead
HAC (Smile)	82.4	100.0%	Minimal
CluStream (MOA)	3,854.2	82.5%	High (Init)
ClusTree (MOA)	2,510.8	78.1%	Moderate

The significantly higher latency of CluStream is attributed to the framework initialization cost and the overhead of micro-clustering management relative to the small N per window. Furthermore, micro-clustering summaries “blur” person-specific appearance features, leading to identity merges in dense crowds where HAC maintains distinct clusters. Consequently, HAC is the only viable candidate for maintaining the functional parity reported in Table 3.

6.5. System Benchmark: Architectural Speedup

We benchmark the fundamental performance of the proposed streams-and-tables architecture (**Ours-Base**) against the conventional batch-oriented baseline (**Theirs**). The evaluation is conducted across two datasets: the Woven VisionAI WTS dataset [42] and the NVIDIA SmartSpaces dataset [43], allowing us to observe scaling effects under different data densities.

Table 5 summarizes the architectural speedup achieved for the primary Multi-Camera People Tracking (MCPT) stages. Our implementation significantly outperforms the baseline, particularly in similarity matrix generation, which forms the primary computational bottleneck in the original version.

Table 5: Architectural Speedup (**Theirs** vs. **Ours-Base**) for Woven WTS and NVIDIA SmartSpaces Datasets (Average Latency per Window over a 50-window stress test).

Pipeline Stage	Woven WTS Dataset			NVIDIA SmartSpaces Dataset		
	Theirs (ms)	Ours (ms)	Speedup	Theirs (ms)	Ours (ms)	Speedup
Re-ID Similarity Matrix	1260.00	49.27	25.57$\times$	1671.60	10.67	156.6$\times$
Total MCPT Latency	2210.70	515.99	4.28$\times$	9581.12	280.65	34.1$\times$

The marked disparity in speedup values—4.28 $\times$ for Woven WTS vs. 34.1 $\times$ for NVIDIA SmartSpaces—is directly attributable to the varying **tracklet density** of the two scenes. In the Woven WTS Dataset, the scene is relatively sparse, with an average of **192 tracklets** distributed across **4 cameras**. Here, the baseline’s bottleneck is less pronounced, resulting in a moderate latency of 2.2 seconds.

In contrast, the NVIDIA SmartSpaces dataset represents a high-density “stress test” environment, peaking at over **556 tracklets** in the global network across **4 cameras**. As the density increases from 192 to 556 tracklets (a 2.9 $\times$ increase), the baseline’s workload increases dramatically, with average per-window latency reaching **9.6 seconds**. Meanwhile, our relational streaming engine remains stable at sub-300ms average latencies. This demonstrates that **STREAMTRACK** effectively neutralizes the “state explosion” that occurs in dense crowds, maintaining a stable, real-time pace where batch-oriented approaches fail.

6.6. Topological Optimization: NVIDIA SmartSpaces Dataset

While the streaming architecture provides a significant baseline improvement, the computational complexity still scales with the number of tracklets in the aggregation window. By introducing *Topology-Aware Stream Joins*, we further optimize the association search space by pruning the join predicates using camera proximity. Figure 6 visualizes this approach, showing the spatial relationship of localized tracking groups.



Figure 6: Topological partitioning of the NVIDIA SmartSpaces network. Four topology-aware groups are shown: Group 1 (Cameras 02, 17—blue), Group 2 (Cameras 01, 11—green), Group 3 (Cameras 19, 27—orange), and Group 4 (Cameras 21, 25, 30—purple). Cameras within each group share overlapping fields of view and are joined as independent stream topologies.

To quantify this optimization, we expanded the test environment to include 9 cameras (01, 02, 11, 17, 19, 21, 25, 27, 30) from the NVIDIA SmartSpaces dataset and compare the **Global** (single-group) execution against the **Topology-Aware** (4-group) partitioning. Table 6 summarizes the results.

Table 6: Impact of Topological Optimization: Global vs. Topology-Aware (NVIDIA SmartSpaces).

Metric	Global (1 Group)	Topology-Aware (4 Groups)	Speedup Factor
Avg. MCPT Latency	41.96 ms	10.10 ms	4.16×

The results show that partitioning the NVIDIA SmartSpaces network into four topological groups reduces the average MCPT clustering latency by **4.16×**, from 41.96 ms to 10.10 ms per association call.

This performance gain is primarily driven by reducing the number of tracklets processed per clustering call, localizing the workload to each topological neighborhood.

6.7. Runtime Topology Reconfiguration (RQ3)

A distinctive operational advantage of **STREAMTRACK** is its ability to reconfigure the camera network topology at runtime—without any pipeline downtime, restart, or loss of tracking state. Because camera neighborhoods are maintained as a relational `CameraTopologyTable` rather than hardcoded logic, operators can add, remove, or reassign camera links by simply injecting `CameraTopology` events into the live stream. The system’s reactive reconciliation loop automatically detects the change, recomputes the Connected Components of the neighborhood graph, and atomically deploys the updated EPL aggregation queries.

- **Zero Downtime:** The pipeline continues processing without interruption. No restart, no pause, no manual intervention.
- **Instant Propagation:** The new topology takes effect on the *very next* synchronization window (sub-second propagation latency).
- 495 • **Scoped Impact:** Only the affected camera groups are redeployed; unrelated groups remain completely untouched.
- **State Preservation:** The `GlobalIDTable` and all accumulated tracking state are fully preserved across the reconfiguration boundary.

We validated this capability on the NVIDIA SmartSpaces network. The pipeline was initialized
 500 with the four-group topology shown in the pre-reconfiguration state of Figure 7 and processed two synchronization windows (frames 1–180). At frame 180, a reconfiguration event was injected—*without pausing or restarting the pipeline*—executing two simultaneous changes: (i) a **group split** (disabling the link between cameras 01 and 11) and (ii) a **group merge** (enabling a new link between cameras 17 and 19, merging two groups into a single four-camera cluster). Figure 7 visualizes this reconfiguration
 505 transition on the actual floor plan.

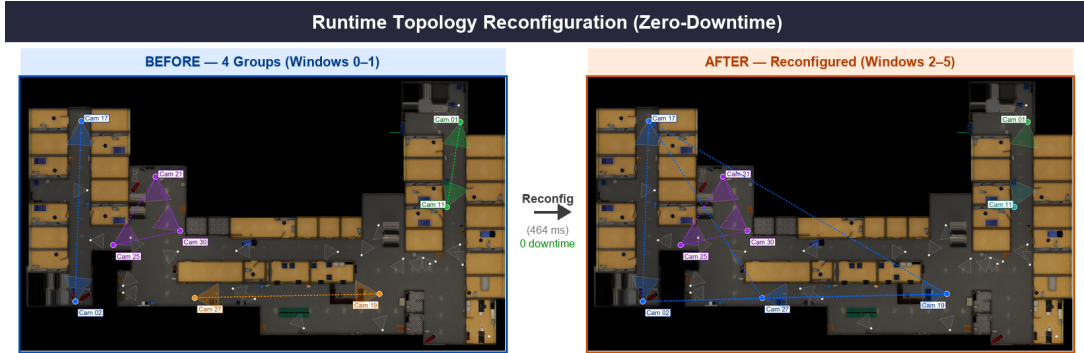


Figure 7: Runtime topology reconfiguration on the NVIDIA SmartSpaces floor plan. The left panel shows the initial four-group topology, and the right panel shows the reconfigured state after a group split (cameras 01 and 11) and a group merge (cameras 17 and 19). The transition completed in **464 ms** with **zero downtime** or interruptions to the tracking pipeline.

Quantitative Results.. The entire reconfiguration cycle completed in **464 ms**—a negligible overhead relative to the 3-second synchronization window—with zero pipeline interruptions and zero post-reconfiguration errors. Following the transition, the pipeline successfully processed the subsequent windows (Windows 2–5) under the new topology, preserving all tracking states while handling up to **58 tracklets** in the
 510 newly-merged four-camera group.

6.8. Overall Performance Analysis

The benchmarking results across 50 data windows demonstrate that **STREAMTRACK** provides not only a reduction in single-window latency but also a massive improvement in total system throughput.

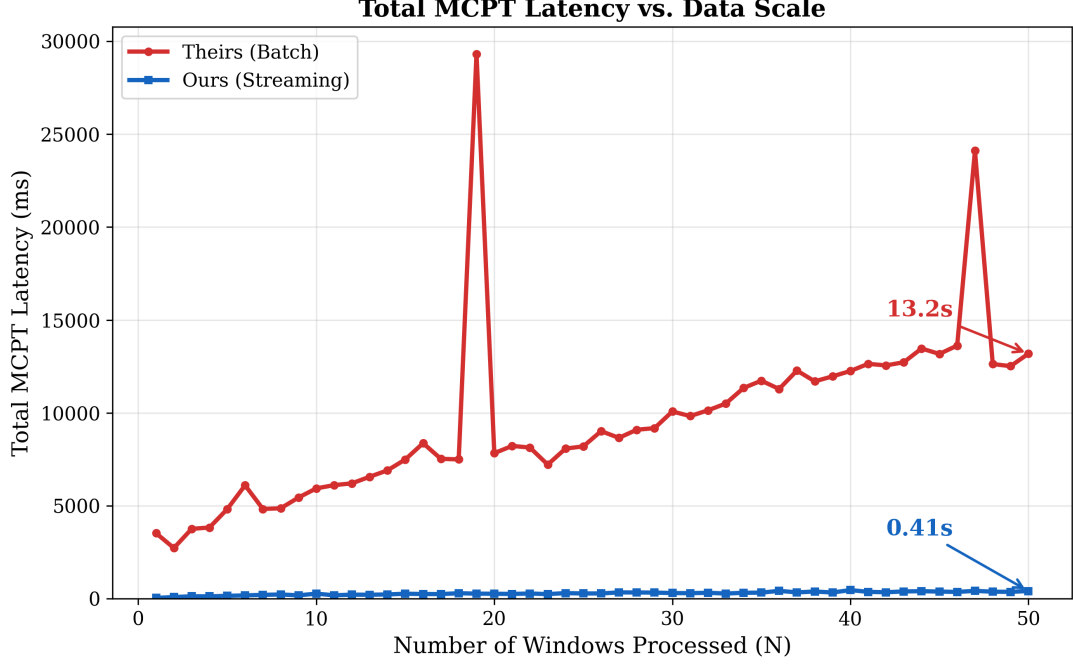


Figure 8: Total MCPT Latency comparison over 50 windows. The batch baseline (**Theirs**) scales quadratically, reaching over 13 seconds. In contrast, the relational streaming architecture (**Ours**) processes the identical workload in approximately 407 ms.

For the NVIDIA SmartSpaces dataset, our system achieved a **34×** average speedup in total MCPT latency, with the core similarity matrix generation being over **156×** faster on average than the baseline. At peak density (window 50), the speedup reaches over **210×** for similarity generation alone. Most significantly, while the baseline’s execution time begins to deviate sharply as the tracklet density grows, our relational streaming architecture maintains a sub-second processing pace for the entire 50-window run. This confirms the system’s ability to maintain high-velocity tracking in large-scale multi-camera environments where batch-oriented approaches fail to meet real-time constraints.

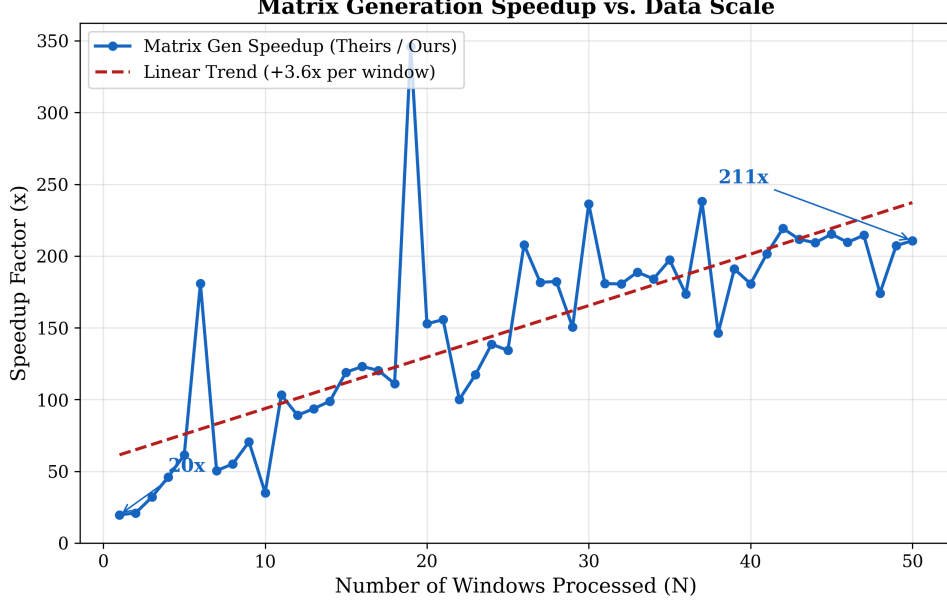


Figure 9: Speedup factor of the relational streaming architecture relative to the batch baseline. The performance gain increases significantly with tracklet density, demonstrating the scalability of the stream-oriented approach.

Figure 9 visualizes this divergence. As the number of processed windows increases and the tracklet density grows toward its peak of 556, the performance advantage of the streaming architecture grows from roughly 20 $\times$ to over 210 $\times$ for the similarity matrix stage alone. This confirms that **STREAMTRACK** effectively neutralizes the scalability bottleneck that plagues conventional batch-oriented MCPT implementations.

The results confirm that **STREAMTRACK** achieves a high-performance balance between latency and scalability. The transition to a **relational streams-and-tables architecture (Ours-Base)** removes the significant bottlenecks associated with similarity matrix generation by leveraging incremental windowing and continuous state management. While batch-oriented pipelines like **Theirs** are forced to compare all person-hypotheses globally, our system exploits the sparsity of real-world camera networks.

By defining association as a join over a reconfigurable **CameraTopologyTable**, we eliminate the "search space explosion" that occurs in conventional MCOT implementations. This makes **STREAMTRACK** uniquely suited for large-scale urban deployments where the number of cameras can reach the thousands, yet each camera cluster remains small enough for sub-10ms processing. The bottleneck in our system is shifted from algorithmic complexity to local streaming I/O, providing a predictable and stable "Real-Time" guarantee that remains consistent even as the global camera network expands.

The breakthrough in *Similarity Matrix Generation* (156 $\times$ average speedup, peaking at 210 $\times$ on the NVIDIA SmartSpaces dataset) is specifically attributed to the architectural shift from batch-oriented list processing to **stream-oriented relational operations** which enable efficient memory locality and incremental updates. At the implementation level, this is further accelerated by the JVM's Just-In-Time (JIT) compilation of compiled primitive floating-point loops, which optimizes loop execution and avoids the boxing/unboxing and interpreter overhead of the dynamically typed reference baseline. Overall, the system achieves a processing time of 516 milliseconds for a 4-camera window in the Woven WTS dataset,

demonstrating its robust capability for real-time operation in diverse multi-camera environments.

7. Discussion

7.1. Benefits of a SQL/Streams-and-Tables Formulation

Expressing MCOT as continuous queries provides significant operational benefits. **Inspectability:** Developers can query the `GlobalIDTable` at runtime using standard SQL to audit association decisions and trace identity assignments. **Modularity:** New vision operators (e.g., pose estimation) can be integrated by adding new join predicates without modifying the core system architecture. **Reconfigurability:** Camera neighborhoods can be updated via streams without restarting the pipeline.

7.2. Limitations and Failure Modes

Despite the high performance, certain challenges remain. Extremely crowded scenes may still lead to similarity ambiguities that even topological gating cannot resolve. Furthermore, rapid lighting changes or occlusions can cause embedding drift, requiring more sophisticated online re-training of the appearance models—a direction for future work.

7.3. Deployment Considerations

The streaming architecture naturally supports edge-to-cloud partitioning. Since the single-camera tracking stage (SCPT) operates independently per camera and emits compact `SingleCameraResult` events, it can be deployed at the edge, while the topology-aware joins and global clustering are centralized in a CEP engine. Our benchmarks show that the MCPT clustering stage processes a 4-camera window in 516ms (Woven WTS) and a topology-aware group averaging 10.10ms per call (Table 6), confirming that even edge devices with modest compute can handle the centralized association workload. This partitioning reduces backhaul bandwidth from raw video streams to lightweight feature events while maintaining a global view of person identities.

8. Related Work

Multi-Camera People Tracking: AI City Challenge Baselines.. The NVIDIA AI City Challenge [15] has catalyzed high-performing multi-camera people tracking (MCPT) systems. In the 2024 edition (Track 1), the highest-scoring offline entry by Yoshida et al. [4] (RIIPS/Yachiyo, achieving the highest raw HOTA of 71.94%) proposed an *offline* framework built on a custom *Overlap Suppression Clustering* (OSC) algorithm. OSC leverages future-frame context to retrospectively resolve overlap anomalies—cases where a single tracklet erroneously contains detections from multiple identities occupying the same frame—using weighted degree centrality for bipartite re-assignment. For cross-camera Re-ID, they categorize detections into three confidence levels (Lv.3/2/1) based on HRNet pose estimation keypoints, select representative images from the highest available level, and cluster them via average-linkage hierarchical clustering on a similarity matrix refined by world-coordinate Euclidean distances. Low-identifiability tracklets are assigned to existing clusters as a final post-processing step. Our system takes this exact pipeline as the

functional baseline and re-architects it as a streaming program, achieving 100% Global ID parity and $< 10^{-6}$ MAD in similarity matrices *online*, without requiring offline post-processing.

Among online challenge entrants, Kim et al. [5] (Nota, 3rd place, with a raw HOTA score of 60.93%) proposed *Cluster Self-Refinement* (CSR), which periodically executes agglomerative clustering to split tracklets contaminated with mixed-identity features, and merges duplicate global IDs by comparing spatial-visual distances. They further leverage RTMPose for angle-aware ground-plane projection (updating camera-specific interpolation ratios via exponential moving average) and filter occluded bounding boxes via Procrustes shape analysis to ensure diverse, high-quality appearance features in each tracklet. Xie et al. [6] (SJTU-Lenovo, 2nd place in raw HOTA score of 67.22% but the overall Track 1 winner after the 10% online tracking bonus) combined geometric consistency constraints with state-aware Re-ID correction to handle camera handoffs. Specker [17] (OCMCTrack, 4th place, with a raw HOTA score of 60.88%) introduced a corrective matching cascade consisting of track splitting (to prune false-positive associations based on visual similarity thresholds), position-based hierarchical clustering, adapted projection-line matching (to handle bounding boxes truncated at frame borders), and Re-ID-based rematching for lost tracks. Yang et al. [18] (UW-ETRI, 5th place, raw HOTA score of 57.14%) decoupled spatial association (clustering multi-camera foot-point projections) from temporal tracking (world-coordinate Kalman filtering), but relied heavily on offline post-processing—tracklet splitting via inner cosine feature distance and hierarchical merging—to correct cumulative online errors.

Unlike these approaches, **STREAMTRACK** does not employ global per-frame bipartite matching cascades or separate offline correction passes. Instead, we express the association logic as continuous SQL/EPL queries over relational state tables (`GlobalIDTable`, `CameraTopologyTable`). Identity switches and stale tracks are resolved incrementally via priority-ordered EPL timer queries, and the search space is pruned from global $O(N^2)$ comparisons to $O(N)$ locally via topology-aware stream joins—preserving absolute functional accuracy while delivering a $34\times$ overall speedup.

Online vs. Offline Tracking and Scalability.. Multi-camera tracking systems are broadly classified as *offline* (batch) or *online* (incremental). Offline methods [4] leverage global optimization over entire video sequences, exploiting future frames for higher accuracy but at latencies that preclude real-time deployment. Online methods [5, 6, 17] process observations causally, yet even these “online” solutions typically rely on internal batch windows for cross-camera association, accumulating detections over fixed intervals before performing global clustering. Critically, *scalability* in the MCPT literature is almost exclusively treated as a computer-vision concern (handling denser crowds, more detections) rather than a *systems* concern (adding cameras, reconfiguring topology, maintaining state under growth). Our work is orthogonal to these accuracy-centric advances: we take an existing offline pipeline and re-architect it as a streaming program, preserving its functional output while gaining systems-level properties—scalability via topology-aware partitioning ($4.16\times$ latency reduction at 9 cameras), inspectability via queryable state tables, and reconfigurability via live topology updates (464 ms reconfiguration with zero downtime).

Industrial Video Analytics Frameworks.. NVIDIA DeepStream [44] is the dominant industrial SDK for GPU-accelerated video analytics. Built on GStreamer, it provides a modular pipeline of hardware-

accelerated plugins for decoding, inference, and single-camera tracking (NvDCF, DeepSORT, IOU). However, its native trackers operate within a single camera stream; cross-camera identity association requires an external microservice layer. NVIDIA Metropolis [16] addresses this gap with a Kubernetes-based reference architecture that decomposes multi-camera tracking into containerized microservices communicating via message brokers. While Metropolis provides a scalable deployment model, it remains a heavyweight, opaque infrastructure: association logic is embedded in compiled services rather than expressed as declarative queries, intermediate tracking state is not directly queryable for debugging, and reconfiguring the camera network typically requires container re-deployment or manual API calls rather than a single streaming event. In contrast, **STREAMTRACK** expresses association as parameterized SQL/EPL joins over a relational topology table, enabling zero-downtime reconfiguration, SQL-based state inspection, and a lightweight single-process deployment.

Stream Processing Foundations and CEP Engine Selection.. Our architectural approach draws from the streams-and-tables duality formalized by Sax et al. [7]. Apache Kafka [34] and its Kafka Streams library operationalized this duality via KTables and changelog topics, enabling stateful stream processing at scale, though without native support for the windowed join semantics and complex event sequence patterns required by vision workloads. Apache Flink [33] provides a more expressive model with native CEP pattern detection, event-time watermarks, and exactly-once guarantees, but introduces significant infrastructure complexity (cluster deployment, checkpoint management) and network serialization overhead when transferring high-dimensional visual feature tensors across distributed worker nodes. Furthermore, Flink’s static operator graph makes zero-downtime query redeployment—required when the camera topology changes—extremely costly, as it necessitates stopping the job, modifying the DAG, and restarting from a savepoint.

Our implementation uses Esper [32], a mature CEP engine that combines SQL-like continuous queries (EPL) with full relational table semantics in an embeddable, single-process JVM runtime. By running within the application process, Esper enables zero-overhead sharing of heavy appearance feature tensors between tracking operators and state tables, avoiding the serialization/deserialization costs inherent in distributed engines. Esper’s API further supports compiling, deploying, and undeploying EPL statements at runtime, which directly enables our reactive topology reconfiguration loop.

The closest prior work applying CEP to video analytics is VidCEP [45], which introduced a graph-based Video Event Knowledge Graph (VEKG) and a Video Event Query Language (VEQL) for detecting spatiotemporal patterns (e.g., “Red Car Left of Person”, loitering, traffic flow) in video streams. As analyzed in VidCEP, existing Event Query Languages (such as Snoop [38], SASE [39], TESLA [40], and Cayuga [46]) are designed for temporal sequence matching on alphanumeric event streams and lack the ability to process high-dimensional video streams or reason about spatial relationships. On the other hand, database-oriented video query languages (such as VSQL [47], MOQL [35], BilVideo [36], SVQL [37], and CVQL [48]) support object, attribute, and spatial relationship queries, but are strictly built for batch querying of offline, pre-annotated databases, failing to handle continuous, real-time streaming joins. VidCEP addresses this by extending Esper with external deep learning frameworks to populate a graph-based

event model. However, VidCEP targets *post-tracking event and anomaly detection*—recognizing high-level behavior patterns from already-segmented and tracked object trajectories—rather than the structurally distinct problem of *multi-camera identity tracking (MCOT)*. MCOT requires continuously maintaining a mutable table of active identity mappings (`GlobalIDTable`), resolving coordinate projections, and executing topology-aware joins in real time. We choose Esper because it serves as a high-performance relational coordinator: we do not attempt to represent raw pixels or neural network inference inside the declarative query language itself; rather, we offload vision kernels (Re-ID, projection) to modular operators and use Esper to orchestrate stream windowing, relational state maintenance, and topological gating. To our knowledge, no prior work has applied the streams-and-tables paradigm to the core MCOT pipeline itself.

Summary and Gap.. The literature reveals a clear gap at the intersection of multi-camera tracking and stream processing systems. CV-oriented works [4, 5, 6, 17, 18] focus on improving tracking heuristics but treat the processing architecture as a fixed implementation detail. Industrial frameworks [44, 16] provide scalable deployment but lack declarative association logic, queryable state, and zero-downtime reconfigurability. Stream processing systems [7, 33, 34] and video CEP frameworks [45] provide theoretical and practical foundations for continuous data processing but have not been applied to the core MCOT pipeline. **STREAMTRACK** bridges this gap by formulating multi-camera tracking as a relational streaming program in which identity is a continuously maintained table, association is a parameterized stream join, and the camera topology is a reconfigurable state relation.

9. Conclusion and Future Work

We presented **STREAMTRACK**, a relational streams-and-tables architecture for multi-camera object tracking. By recasting MCOT as a continuous query problem and utilizing topology-aware stream joins, we achieved a **156× average speedup** in similarity generation and a **34× overall speedup** over batch-oriented baselines on the NVIDIA SmartSpaces dataset. Scaling the NVIDIA SmartSpaces network with topology-aware partitioning yields a further **4.16×** latency reduction per association call. Our system ensures real-time performance even as the camera network grows, while providing an architecture that supports unique operational benefits like runtime reconfigurability and SQL-based inspectability.

Future work.. Future directions include: (i) multi-hop topology reasoning, where cameras beyond immediate neighbors contribute to association via transitive similarity; and (ii) learning topology weights from historical data to dynamically adjust join selectivity.

References

- [1] Y. Qian, L. Yu, W. Liu, A. G. Hauptmann, Electricity: An efficient multi-camera vehicle tracking system for intelligent city, in: 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW), 2020, pp. 2511–2519. doi:10.1109/CVPRW50498.2020.00302.

- [2] D. Thirde, M. Borg, J. Ferryman, J. Aguilera, M. Kampel, G. Fernandez, Multi-camera tracking for visual surveillance applications, in: Proceedings of the Computer Vision Winter Workshop (CVWW), Czech Pattern Recognition Society, 2006.
- [3] M. K. Bhuyan, B. C. Lovell, A. Bigdeli, Tracking with multiple cameras for video surveillance, in: 2007 International Conference on Digital Image Computing: Techniques and Applications (DICTA), 2007, pp. 592–599. doi:10.1109/DICTA.2007.4426852.
- [4] R. Yoshida, J. Okubo, J. Fujii, M. Amakata, T. Yamashita, Overlap suppression clustering for offline multi-camera people tracking, in: 2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW), Seattle, WA, USA, 2024, pp. 7153–7162. doi:10.1109/CVPRW63382.2024.00710.
- [5] J. Kim, W. Shin, H. Park, D. Choi, Cluster self-refinement for enhanced online multi-camera people tracking, in: 2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW), Seattle, WA, USA, 2024, pp. 7190–7197. doi:10.1109/CVPRW63382.2024.00714.
- [6] Z. Xie, Z. Ni, W. Yang, Y. Zhang, Y. Chen, Y. Zhang, X. Ma, A robust online multi-camera people tracking system with geometric consistency and state-aware re-id correction, in: 2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW), Seattle, WA, USA, 2024, pp. 7007–7016. doi:10.1109/CVPRW63382.2024.00694.
- [7] M. Sax, G. Wang, M. Weidlich, J. Freytag, Streams and tables: Two sides of the same coin, in: BIRTE, ACM, Rio de Janeiro, Brazil, 2018, pp. 1–10. doi:10.1145/3242153.3242155. URL <https://doi.org/10.1145/3242153.3242155>
- [8] A. Arasu, S. Babu, J. Widom, CQL: A language for continuous queries over streams and relations, in: DBPL, Springer, Berlin, 2003, pp. 1–19. doi:10.1007/978-3-540-24607-7_1. URL https://doi.org/10.1007/978-3-540-24607-7_1
- [9] A. Awad, F. Awaysheh, H. A. López, BEST: a unified business process enactment via streams and tables for service computing, Process Sci. 2 (1). doi:10.1007/S44311-025-00030-8. URL <https://doi.org/10.1007/s44311-025-00030-8>
- [10] A. Bewley, Z. Ge, L. Ott, F. Ramos, B. Upcroft, Simple online and realtime tracking, in: 2016 IEEE International Conference on Image Processing (ICIP), 2016, pp. 3464–3468. doi:10.1109/ICIP.2016.7533003.
- [11] N. Wojke, A. Bewley, D. Paulus, Simple online and realtime tracking with a deep association metric, in: 2017 IEEE International Conference on Image Processing (ICIP), 2017, pp. 3645–3649. doi:10.1109/ICIP.2017.8296962.
- [12] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, X. Wang, Bytetrack: Multi-object tracking by associating every detection box, in: Computer Vision – ECCV 2022:

17th European Conference, Springer-Verlag, Berlin, Heidelberg, 2022, pp. 1–21. doi:10.1007/978-3-031-20047-2_1.

- [13] M. Ye, J. Shen, G. Lin, T. Xiang, L. Shao, S. C. H. Hoi, Deep learning for person re-identification: A survey and outlook, *IEEE Transactions on Pattern Analysis and Machine Intelligence* 44 (6) (2021) 2872–2893. doi:10.1109/TPAMI.2021.3054775.

- [14] E. Ristani, F. Solera, R. Zou, R. Cucchiara, C. Tomasi, Performance measures and a data set for multi-target, multi-camera tracking, in: *Computer Vision – ECCV 2016 Workshops*, Vol. 9914 of *Lecture Notes in Computer Science*, Springer, Cham, 2016, pp. 17–35. doi:10.1007/978-3-319-48881-3_2.

- [15] S. Wang, D. C. Anastasiu, Z. Tang, M.-C. Chang, Y. Yao, L. Zheng, M. S. Rahman, M. S. Arya, A. Sharma, P. Chakraborty, S. Prajapati, Q. Kong, N. Kobori, M. Gochoo, M.-E. Otgonbold, F. Alnajjar, G. Batnasan, P.-Y. Chen, J.-W. Hsieh, X. Wu, S. S. Pusegaonkar, Y. Wang, S. Biswas, R. Chellappa, The 8th AI city challenge, in: *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, Seattle, WA, USA, 2024.

- [16] NVIDIA Corporation, NVIDIA Metropolis – multi-camera tracking AI workflow, <https://docs.nvidia.com/metropolis/>, microservices Reference Architecture 2.1. Accessed: 2026-06-18 (2025).

- [17] A. Specker, OCMCTrack: Online multi-target multi-camera tracking with corrective matching cascade, in: *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, Seattle, WA, USA, 2024, pp. 7236–7244. doi:10.1109/CVPRW63382.2024.00719.

- [18] C.-Y. Yang, H.-W. Huang, P.-K. Kim, Z. Jiang, K.-J. Kim, C.-I. Huang, H. Du, J.-N. Hwang, An online approach and evaluation method for tracking people across cameras in extremely long video sequence, in: *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, Seattle, WA, USA, 2024, pp. 7037–7045. doi:10.1109/CVPRW63382.2024.00697.

- [19] A. Awad, R. Tommasini, S. Langhi, M. Kamel, E. D. Valle, S. Sakr, D²ia: User-defined interval analytics on distributed streams, *Inf. Syst.* 104 (2022) 101679. doi:10.1016/J.IS.2020.101679. URL <https://doi.org/10.1016/j.is.2020.101679>

- [20] N. Giatrakos, E. Alevizos, A. Artikis, A. Deligiannakis, M. N. Garofalakis, Complex event recognition in the big data era: a survey, *VLDB J.* 29 (1) (2020) 313–352. doi:10.1007/s00778-019-00557-W. URL <https://doi.org/10.1007/s00778-019-00557-w>

- [21] F. M. Awaysheh, M. Alazab, S. Garg, D. Niyato, C. Verikoukis, Big data resource management & networks: Taxonomy, survey, and future directions, *IEEE Communications Surveys & Tutorials* 23 (4) (2021) 2098–2130.

- [22] A. Arasu, S. Babu, J. Widom, The CQL continuous query language: semantic foundations and query execution, *VLDB J.* 15 (2) (2006) 121–142. doi:10.1007/s00778-004-0147-Z.

URL <https://doi.org/10.1007/s00778-004-0147-z>

- [23] J. Verwiebe, P. M. Grulich, J. Traub, M. Volker, Survey of window types for aggregation in stream processing systems, *The VLDB Journal* (2023) 1–27.
- [24] H. Jafarpour, R. Desai, KSQL: streaming SQL engine for apache kafka, in: M. Herschel, H. Galhardas, B. Reinwald, I. Fundulaki, C. Binnig, Z. Kaoudi (Eds.), *Advances in Database Technology - 22nd International Conference on Extending Database Technology, EDBT 2019, Lisbon, Portugal, March 26-29, 2019, OpenProceedings.org, Lisbon, Portugal, 2019*, pp. 524–533. doi:10.5441/002/EDBT.2019.48.
URL <https://doi.org/10.5441/002/edbt.2019.48>
- [25] A. Artikis, A. Margara, M. Ugarte, S. Vansummeren, M. Weidlich, Complex event recognition languages: Tutorial, in: *Proceedings of the 11th ACM International Conference on Distributed and Event-based Systems, DEBS 2017, Barcelona, Spain, June 19-23, 2017, ACM, New York, NY, USA, 2017*, pp. 7–10. doi:10.1145/3093742.3095106.
URL <https://doi.org/10.1145/3093742.3095106>
- [26] A. Arasu, B. Babcock, S. Babu, M. Datar, K. Ito, R. Motwani, I. Nishizawa, U. Srivastava, D. Thomas, R. Varma, J. Widom, STREAM: the stanford stream data manager, *IEEE Data Eng. Bull.* 26 (1) (2003) 19–26.
URL <http://sites.computer.org/debull/A03mar/paper.ps>
- [27] A. Alves, R. J. Smith, L. Williams, *Getting Started with Oracle Event Processing 11g*, Packt Publishing Birmingham, Birmingham, UK, 2013.
- [28] Apache Flink, Apache flink: Stateful computations over data streams, <https://flink.apache.org/>, accessed: 2025-08-15 (2025).
- [29] M. Armbrust, T. Das, J. Torres, B. Yavuz, S. Zhu, R. Xin, A. Ghodsi, I. Stoica, M. Zaharia, Structured streaming: A declarative API for real-time applications in apache spark, in: G. Das, C. M. Jermaine, P. A. Bernstein (Eds.), *Proceedings of the 2018 International Conference on Management of Data, SIGMOD Conference 2018, Houston, TX, USA, June 10-15, 2018, ACM, New York, NY, USA, 2018*, pp. 601–613. doi:10.1145/3183713.3190664.
URL <https://doi.org/10.1145/3183713.3190664>
- [30] S. A. Noghabi, K. Paramasivam, Y. Pan, N. Ramesh, J. Bringham, I. Gupta, R. H. Campbell, Samza: stateful scalable stream processing at linkedin, *Proc. VLDB Endow.* 10 (12) (2017) 1634–1645. doi:10.14778/3137765.3137770.
URL <https://doi.org/10.14778/3137765.3137770>
- [31] Apache Storm, Apache storm, <https://storm.apache.org/>, accessed: 2025-08-15 (2025).
- [32] E. Inc., Esper – complex event processing engine, <https://www.espertech.com/esper/>, version 9.x. Accessed: 2026-03-13 (2024).

- [33] P. Carbone, A. Katsifodimos, S. Ewen, V. Markl, S. Haridi, K. Tzoumas, Apache Flink: Stream and batch processing in a single engine, *IEEE Data Engineering Bulletin* 38 (4) (2015) 28–38.
- [34] J. Kreps, N. Narkhede, J. Rao, Kafka: A distributed messaging system for log processing, in: *Proceedings of the Workshop on Networking Meets Databases (NetDB)*, 2011.
- [35] J. Z. Li, M. T. Oszu, D. Szafron, V. Oria, MOQL: A multimedia object query language, in: *Proceedings of the 3rd International Workshop on Multimedia Information Systems*, 1997, pp. 115–124.
- [36] M. E. Donderler, E. Saykol, U. Arslan, O. Ulusoy, U. Gudukbay, BilVideo: Design and implementation of a video database management system, *Multimedia Tools and Applications* 27 (1) (2005) 79–104.
- [37] C. Lu, M. Liu, Z. Wu, SVQL: A SQL extended query language for video databases, *International Journal of Database Theory and Application* 8 (4) (2015) 109–118.
- [38] S. Chakravarthy, D. Mishra, Snoop: An expressive event specification language for active databases, *Data & Knowledge Engineering* 13 (3) (1994) 301–326.
- [39] E. Wu, Y. Diao, S. Rizvi, High-performance complex event processing over streams, in: *Proceedings of the 2006 ACM SIGMOD international conference on Management of data (SIGMOD)*, 2006, pp. 407–418.
- [40] G. Cugola, A. Margara, TESLA: a formally defined event specification language, in: *Proceedings of the Fourth ACM International Conference on Distributed Event-Based Systems (DEBS)*, 2010, pp. 50–61.
- [41] A. Neubeck, L. Van Gool, Efficient non-maximum suppression, in: *18th International Conference on Pattern Recognition (ICPR’06)*, Vol. 3, IEEE, 2006, pp. 850–855. doi:10.1109/ICPR.2006.479.
- [42] Q. Kong, Y. Kawana, R. Saini, A. Kumar, J. Pan, T. Gu, Y. Ozao, B. Opra, D. C. Anastasiu, Y. Sato, N. Kobori, Wts: A pedestrian-centric traffic video dataset for fine-grained spatial-temporal understanding, homepage: <https://woven-visionai.github.io/wts-dataset-homepage/> (2024). arXiv:2407.15350.
- URL <https://arxiv.org/abs/2407.15350>
- [43] NVIDIA, A. Challenge, Ai city challenge 2025 track 1: Multi-camera people tracking, <https://www.aicitychallenge.org/2025-track1/>, data hosted at <https://huggingface.co/datasets/nvidia/PhysicalAI-SmartSpaces>. Accessed: 2026-03-13 (2025).
- [44] NVIDIA Corporation, NVIDIA DeepStream SDK – streaming analytics toolkit for AI-based multi-sensor processing, <https://developer.nvidia.com/deepstream-sdk>, version 7.x. Accessed: 2026-06-18 (2024).
- [45] P. Yadav, E. Curry, VidCEP: Complex event processing framework to detect spatiotemporal patterns in video streams, in: *2019 IEEE International Conference on Big Data (Big Data)*, 2019, pp. 2513–2522. doi:10.1109/BigData47090.2019.9006018.

- 825 [46] A. Demers, J. Gehrke, B. Panda, M. Riedewald, V. Sharma, W. White, Cayuga: A general purpose
event monitoring system, in: Proceedings of the Third Biennial Conference on Innovative Data
Systems Research (CIDR), 2007, pp. 412–422.
- [47] W. Aref, M. Hammad, A. C. Catlin, I. Ilyas, T. Ghanem, A. Elmagarmid, M. Marzouk, Video
query processing in the vdbms testbed for video database research, in: Proceedings of the 1st
830 ACM International Workshop on Multimedia Databases, MMDB '03, Association for Computing
Machinery, New York, NY, USA, 2003, p. 25–32. doi:10.1145/951676.951682.
URL <https://doi.org/10.1145/951676.951682>
- [48] T. L. Le, M. Thonnat, A. Boucher, F. Bremond, A query language combining object features and
semantic events for surveillance video retrieval, in: Proceedings of the 14th International Conference
835 on Multimedia Modeling (MMM), 2008, pp. 209–220.